

SÜLEYMAN DEMİREL ÜNİVERSİTESİ



BİL-304 **BİLGİSAYAR MİMARİSİ** **Dr. Öğr. Üyesi Fatih GÖKÇE**

7. Hafta

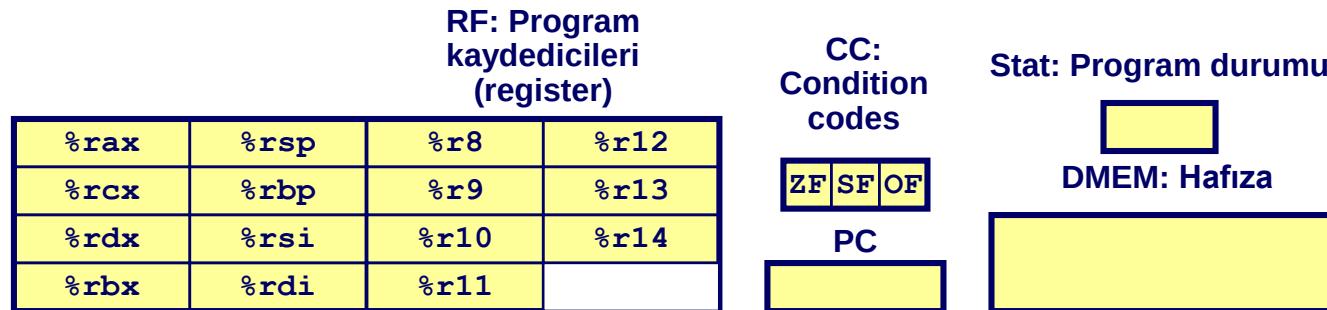
Bölüm / Hafta Hedefleri

- **Komut Seti Mimarisinin 2. Kısımını anlatacağımız bu derste aşağıdaki konu başlıklarının öğrenilmesi hedeflenmektedir:**
 - Y86-64 İşlemcisi Komut Seti,
 - Aritmetik ve Lojik Komutlar,
 - Taşıma Komutları,
 - Hafıza Adresleme Modları

Bu ders notundaki slaytlar ders kitabına ait sunum dosyalarından adapte edilmiştir:
<http://csapp.cs.cmu.edu/>

Adapted from slides of the textbook: <http://csapp.cs.cmu.edu/>

Y86-64 İşlemci Durumu



- Program Kaydedicileri (Registers)
 - 15 kaydedici (%r15 hariç). Her biri 64 bit
- Durum kodları (Condition Codes)
 - Tek bitlik bayraklar aritmetik veya lojik komutlar tarafından set edilir.
 - » ZF: Sıfır SF:Negatiflik OF: İki tümleyen taşma durum
 - CF: Elde biti X86-64'te bulunmaktadır ama Y86-64 işlemcide bulunmaz.
- Program Sayacı (Program Counter)
 - Bir sonraki komutun adresini gösterir
- Program Durumu
 - Normal çalışma veya hata durumlarını belirtir
- Hafıza
 - Byte adresli kayıt dizisi
 - Sözcükler (Words) küçük-sonlu (little-endian) yöntemine göre kaydedilir

Y86-64 Komut Seti #1

Byte	0	1	2	3	4	5	6	7	8	9
halt	0	0								
nop	1	0								
cmovXX rA, rB	2	fn	rA	rB						
irmovq V, rB	3	0	F	rB	V					
rmmovq rA, D(rB)	4	0	rA	rB	D					
mrmmovq D(rB), rA	5	0	rA	rB	D					
OPq rA, rB	6	fn	rA	rB						
jXX Hedef	7	fn	Hedef							
call Hedef	8	0	Hedef							
ret	9	0								
pushq rA	A	0	rA	F						
popq rA	B	0	rA	F						

Y86-64 Komutları

Format

- 1–10 byte arasında bilgi hafızadan okunur
 - Komutun uzunluğu (kaç byte olduğu) ilk byte'ından anlaşılır.
 - x86-64 komutlarından daha basit kodlanırlar ve sayıca daha azdırlar
- Her biri program durumuna erişebilir ve değiştirebilir.

Y86-64 Komut Seti

Byte	0	1	2	3	4	5	6	7	8	9
halt	0	0								
nop	1	0								
cmovXX rA, rB	2	fn	rA	rB						
irmovq V, rB	3	0	F	rB	V					
rmmovq rA, D(rB)	4	0	rA	rB	D					
mrmmovq D(rB), rA	5	0	rA	rB	D					
OPq rA, rB	6	fn	rA	rB						
jXX Hedef	7	fn	Hedef							
call Hedef	8	0	Hedef							
ret	9	0								
pushq rA	A	0	rA	F						
popq rA	B	0	rA	F						

Y86-64 Komut Seti #2

Byte

	0	1	2	3	4	5	6	
halt	0	0						rrmovq 2 0
nop	1	0						cmovle 2 1
cmovXX rA, rB	2	fn	rA	rB				cmovl 2 2
irmovq V, rB	3	0	F	rB			V	cmove 2 3
rmmovq rA, D(rB)	4	0	rA	rB			D	cmovne 2 4
mrmmovq D(rB), rA	5	0	rA	rB			D	cmovge 2 5
OPq rA, rB	6	fn	rA	rB				cmovg 2 6
jXX Hedef	7	fn					Hedef	
call Hedef	8	0					Hedef	
ret	9	0						
pushq rA	A	0	rA	F				
popq rA	B	0	rA	F				

Y86-64 Komut Seti #3

Byte	0	1	2	3	4	5	6	7	8	9				
halt	0	0												
nop	1	0												
cmovXX rA, rB	2	fn	rA	rB										
irmovq V, rB	3	0	F	rB	V									
rmmovq rA, D(rB)	4	0	rA	rB	D									
mrmmovq D(rB), rA	5	0	rA	rB	D									
OPq rA, rB	6	fn	rA	rB	addq 6 0 subq 6 1 andq 6 2 xorq 6 3									
jXX Hedef	7	fn	Hedef											
call Hedef	8	0	Hedef											
ret	9	0												
pushq rA	A	0	rA	F										
popq rA	B	0	rA	F										

Y86-64 Komut Seti #4

Byte	0	1	2	3	4	5	6	7				
halt	0	0								jmp	7 0	
nop	1	0								jle	7 1	
cmovXX rA, rB	2	fn	rA	rB						j1	7 2	
irmovq V, rB	3	0	F	rB	V					je	7 3	
rmmovq rA, D(rB)	4	0	rA	rB	D					jne	7 4	
mrmmovq D(rB), rA	5	0	rA	rB	D					jge	7 5	
OPq rA, rB	6	fn	rA	rB						jg	7 6	
jXX Hedef	7	fn	Hedef									
call Hedef	8	0	Hedef									
ret	9	0										
pushq rA	A	0	rA	F								
popq rA	B	0	rA	F								

Kaydedicilerin (Register) kodlanması

Her kaydedici 4-bit ID'ye sahiptir.

%rax	0	%r8	8
%rcx	1	%r9	9
%rdx	2	%r10	A
%rbx	3	%r11	B
%rsp	4	%r12	C
%rbp	5	%r13	D
%rsi	6	%r14	E
%rdi	7	Kaydedici yok	F

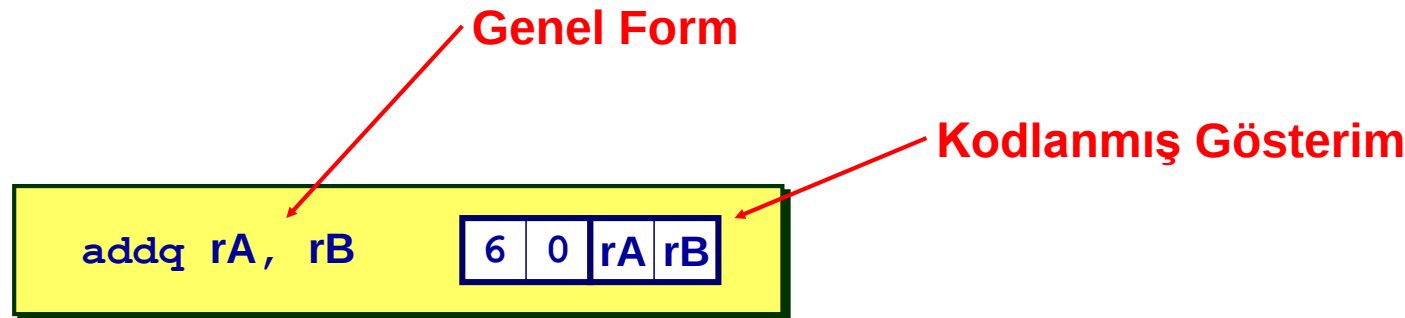
- x86-64'dakiyle aynı kodlama

15 (0xF) ID'si “kaydedici yok” durumunu kodlar

- Bu özelliği donanım tasarımının birçok yerinde kullanacağız.

Komut Örneği

Toplama Komutu



- rA kaydedicisindeki değeri rB kaydedicisindeki değere ekle
 - Sonucu rB'ye kaydet
 - Not: Y86-64 sadece kaydedicilerdeki verilerin toplanmasına imkan sağlar
- Durum kodlarını (condition codes) sonuca bağlı olarak etkiler
- örnek, **addq %rax, %rsi** Kodlanmış hali: **60 06**
- Kodlamasında 2-byte gerekir
 - Birinci byte komut tipini belirtir
 - İkincisi kaynak ve hedef kaydedicileri verir

Aritmetik ve Lojik İşlemler

Komut kodu

Fonksiyon kodu

Topla

`addq rA, rB`

6	0	rA	rB
---	---	----	----

Çıkar (rA'yı rB'den)

`subq rA, rB`

6	1	rA	rB
---	---	----	----

VE

`andq rA, rB`

6	2	rA	rB
---	---	----	----

Özel VEYA (XOR)

`xorq rA, rB`

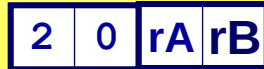
6	3	rA	rB
---	---	----	----

- Genel olarak “OPq” şeklinde ifade edilirler, “OP” yapılan işlemi tanımlayan kısımdır.
- Kodlamaları sadece “fonksiyon kodu” kısmında farklıdır
 - Birinci komut sözcüğünün düşük öncelikli ilk 4 biti
- Durum kodlarını (condition codes) etkilerler

Taşıma (Move) İşlemleri

Kaydedici → Kaydedici

`rrmovq rA, rB`



Sabit değer (Immediate) → Kaydedici

`irmovq V, rB`



V

Kaydedici → Hafıza

`rmmovq rA, D (rB)`



D

Hafıza → Kaydedici

`mrmovq D (rB), rA`



D

- x86-64 `movq` komutuna benzer şekilde çalışır. Kaynak olarak verilen sabit değer, kaydedici içeriği ya da hafıza içeriğini, hedefe kopyalar.
- Hafıza adresleri için daha basit formata sahiptir.
- 4 farklı tipi bulunur ve her biri farklı isimdedir.

Taşıma (Move) Komutu Örnekleri

X86-64

```
movq $0xabcd, %rdx
```

Y86-64

```
irmovq $0xabcd, %rdx
```

Kodlaması: 30 F2 cd ab 00 00 00 00 00 00

```
movq %rsp, %rbx
```

```
rrmovq %rsp, %rbx
```

Kodlaması: 20 43

```
movq -12(%rbp), %rcx
```

```
mrmovq -12(%rbp), %rcx
```

Kodlaması: 50 15 f4 ff ff ff ff ff ff ff

```
movq %rsi, 0x41c(%rsp)
```

```
rmmovq %rsi, 0x41c(%rsp)
```

Kodlaması: 40 64 1c 04 00 00 00 00 00 00

Makine Komut Örnekleri

```
*dest = t;
```

C Kodu

- t değerini dest ile belirtilen yere kaydet

```
movq %rax, (%rbx)
```

Assembly

- 8-byte'lık değeri hafızaya taşı
 - x86-64'deki ifadesiyle dörtlü sözcük (Quad Word)
- İşlenenler (Operands):
 - t: Kaydedici %rax
 - dest: Kaydedici %rbx
 - *dest: Hafıza M[%rbx]

```
0x40059e: 48 89 03
```

Obje Kodu

- 3-byte'lık komut
- 0x40059e adresinde kayıtlıdır

Verinin Taşınması

Verinin Taşınması

`movq Kaynak, Hedef`

İşlenen (Operand) Tipleri

- **Sabit (Immediate):** Sabit integer verisi
 - Örnek: `$0x400`, `$-533`
 - C'deki sabitler gibi, fakat `'$'` öneki ile
 - 1, 2 veya 4 byte ile kodlanır
- **Kaydedici:** 16 integer kaydediciden biri
 - Örnek: `%rax`, `%r13`
 - Fakat `%rsp` özel amaç için ayrılmıştır
 - Diğerleri belli komutlar için özel kullanımlara sahiptir
- **Hafıza:** Kaydedicideki değer ve ofset değeri ile belirtilen hafıza adresinde bulunan 8 art arda byte
 - En basit örneği: `(%rax)`
 - Birçok değişik “adres modu” bulunur

<code>%rax</code>
<code>%rcx</code>
<code>%rdx</code>
<code>%rbx</code>
<code>%rsi</code>
<code>%rdi</code>
<code>%rsp</code>
<code>%rbp</code>
<code>%rN</code>

X86-64 movq İşlenen Kombinasyonları

	Kaynak	Hedef	Kaynak,Hedef	C'de olası karşılığı
movq	Sabit	Kayd.	movq \$0x4,%rax	temp = 0x4;
		Hafıza	movq \$-147, (%rax)	*p = -147;
	Kayd.	Kayd.	movq %rax,%rdx	temp2 = temp1;
		Hafıza	movq %rax, (%rdx)	*p = temp;
	Hafıza	Kayd.	movq (%rax),%rdx	temp = *p;

Hafızadan hafızaya tek komutla veri transferi yapılamaz

Basit hafıza adresleme modları

Normal **(R)** **Mem[Reg[R]]**

- R kaydedicisindeki değer erişilecek hafıza adresini belirtir
- Aha! C'deki Pointer mantığı

```
movq (%rcx) , %rax
```

Ofsetli **D(R)** **Mem[Reg[R]+D]**

- R kaydedicisindeki değer hafıza bölgesinin başlangıcını belirtir
- Erişilecek adres, sabit D ofset değeri eklenerek bulunur

```
movq 8 (%rbp) , %rdx
```



Basit adresleme modu örnekleri

```
void swap
(long *xp, long *yp)
{
    long t0 = *xp;
    long t1 = *yp;
    *xp = t1;
    *yp = t0;
}
```

swap:

```
movq    (%rdi), %rax
movq    (%rsi), %rdx
movq    %rdx, (%rdi)
movq    %rax, (%rsi)
ret
```

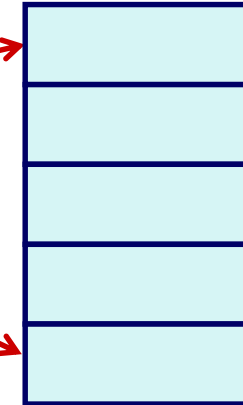
Swap()'in Anlaşılması

```
void swap
(long *xp, long *yp)
{
    long t0 = *xp;
    long t1 = *yp;
    *xp = t1;
    *yp = t0;
}
```

Kaydediciler

%rdi	
%rsi	
%rax	
%rdx	

Hafıza



Kaydedici Değer

%rdi	xp
%rsi	yp
%rax	t0
%rdx	t1

swap:

```
movq    (%rdi), %rax    # t0 = *xp
movq    (%rsi), %rdx    # t1 = *yp
movq    %rdx, (%rdi)    # *xp = t1
movq    %rax, (%rsi)    # *yp = t0
ret
```

Swap()'in Anlaşılması

Kaydediciler

%rdi	0x120
%rsi	0x100
%rax	
%rdx	

Hafıza

Adres	
0x120	123
0x118	
0x110	
0x108	
0x100	456

swap:

```

movq    (%rdi), %rax    # t0 = *xp
movq    (%rsi), %rdx    # t1 = *yp
movq    %rdx, (%rdi)    # *xp = t1
movq    %rax, (%rsi)    # *yp = t0
ret

```

Swap()'in Anlaşılması

Kaydediciler

%rdi	0x120
%rsi	0x100
%rax	123
%rdx	

Hafıza

Adres	
0x120	123
0x118	
0x110	
0x108	
0x100	456



swap:

```

movq    (%rdi), %rax    # t0 = *xp
movq    (%rsi), %rdx    # t1 = *yp
movq    %rdx, (%rdi)    # *xp = t1
movq    %rax, (%rsi)    # *yp = t0
ret

```

Swap()'in Anlaşılması

Kaydediciler

%rdi	0x120
%rsi	0x100
%rax	123
%rdx	456

Hafıza

Adres	
0x120	123
0x118	
0x110	
0x108	
0x100	456



swap:

```

movq    (%rdi), %rax    # t0 = *xp
movq    (%rsi), %rdx    # t1 = *yp
movq    %rdx, (%rdi)    # *xp = t1
movq    %rax, (%rsi)    # *yp = t0
ret

```

Swap()'in Anlaşılması

Kaydediciler

%rdi	0x120
%rsi	0x100
%rax	123
%rdx	456

Hafıza

Adres	
0x120	456
0x118	
0x110	
0x108	
0x100	456

swap:

```

movq    (%rdi), %rax    # t0 = *xp
movq    (%rsi), %rdx    # t1 = *yp
movq    %rdx, (%rdi)    # *xp = t1
movq    %rax, (%rsi)    # *yp = t0
ret

```

Swap()'in Anlaşılması

Kaydediciler

%rdi	0x120
%rsi	0x100
%rax	123
%rdx	456

Hafıza

Adres	
0x120	456
0x118	
0x110	
0x108	
0x100	123



swap:

```

movq    (%rdi), %rax    # t0 = *xp
movq    (%rsi), %rdx    # t1 = *yp
movq    %rdx, (%rdi)    # *xp = t1
movq    %rax, (%rsi)    # *yp = t0
ret

```

Hafıza Adresleme Modlarının Tümü

En Genel Form

$$D(Rb, Ri, S) \quad \text{Mem}[\text{Reg}[Rb] + S * \text{Reg}[Ri] + D]$$

- D: 1, 2 veya 4 byte'lık sabit “offset”
- Rb: Taban kaydedici: 16 integer kaydediciden herhangi biri
- Ri: İndeks kaydedici: %rsp hariç herhangi biri
- S: Ölçek: 1, 2, 4 veya 8 (*Neden bu sayılar?*)

Özel Durumlar

$$(Rb, Ri) \quad \text{Mem}[\text{Reg}[Rb] + \text{Reg}[Ri]]$$

$$D(Rb, Ri) \quad \text{Mem}[\text{Reg}[Rb] + \text{Reg}[Ri] + D]$$

$$(Rb, Ri, S) \quad \text{Mem}[\text{Reg}[Rb] + S * \text{Reg}[Ri]]$$

Adres Hesaplama Örnekleri

<code>%rdx</code>	<code>0xf000</code>
<code>%rcx</code>	<code>0x0100</code>

İfade	Adres Hesabı	Adres
<code>0x8 (%rdx)</code>	<code>0xf000 + 0x8</code>	<code>0xf008</code>
<code>(%rdx, %rcx)</code>	<code>0xf000 + 0x100</code>	<code>0xf100</code>
<code>(%rdx, %rcx, 4)</code>	<code>0xf000 + 4*0x100</code>	<code>0xf400</code>
<code>0x80 (, %rdx, 2)</code>	<code>2*0xf000 + 0x80</code>	<code>0x1e080</code>

2018 Vize / 6. soru

İkili hali **1001101101111100** olan **39804.0** sayısının IEEE 754 standardına göre 32-bitlik float (1 bit işaret biti (s), 8 bit üst (exp) ve 23 bit anlamlı kısım (frac)) olarak ifadesinin hexadecimal (onaltılık) değeri aşağıdakilerden hangisidir?

- A. **0x471B7C00**
- B. **0x08E36F80**
- C. **0x47001B7C**
- D. **0x1B7C008E**

1001101101111100

2018 Vize / 7. soru

Onluk tabanda verilen -9.75 sayısının IEEE 754 standardına göre 11-bitlik float (1 bit işaret biti (s), 4 bit üst (exp) ve 6 bit anlamlı kısım (frac)) olarak ifadesinin hexadecimal (onaltılık) değeri aşağıdakilerden hangisidir?

- A. **0xD87** B. **0x68E** C. **0x40E** D. **0xD1C**

Kaynaklar

- 1. Computer Systems: A Programmer's Perspective, Randal E. Bryant ve David R. O'Hallaron, PEARSON.**

SÜLEYMAN DEMİREL ÜNİVERSİTESİ



TEŞEKKÜRLER